

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	M Vishnu Vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	27-08-2026

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING SIMATS Online Programming Lab DAA PROGRAMMING

CO5 AT2-Pseudocode Writing Task (Algorithm Design)

Language: **Python** C C++ Java

QUESTIONS

Q1 Graph Isomorphism Problem 100 marks

5/7 answered

Q1 Graph Isomorphism Problem 100 marks

Develop a pseudocode using backtracking to check whether two graphs are isomorphic. Clearly identify inputs (two graphs) and output (true/false). Design an algorithm that attempts all possible vertex mappings while preserving edge relationships. Use loops and recursive functions to generate mappings. Apply constraint checking to eliminate invalid mappings. Ensure the pseudocode is structured clearly and logically. Handle all cases and optimize by pruning redundant mappings.

Your Code

```

import java.util.*;

public class GraphIsomorphism {
    static int n;
    static int[][] G1, G2;
    static int[] map;
    static boolean[] used;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        n = sc.nextInt(); // number of vertices (same for both graphs)

        G1 = new int[n][n];
    }

```

Input

```

3
0 1 0
1 0 1
0 1 0

```

Output

```

true - Graphs are isomorphic
Mapping: (0, vertex => 0), (1, vertex => 1), (2, vertex => 2)

```

Step-by-step process

Step 1: Read in both graphs

Take two graphs, G1 and G2. Each is stored as a grid (adjacency matrix) where a 1 means "these two points are connected" and a 0 means "they are not."

Step 2: Check the basic requirement

Both graphs must have the same number of vertices (points). If they don't, they can never be isomorphic — stop immediately.

Step 3: Try to match vertex 0 of G1 to some vertex of G2

Pick an unused vertex in G2 (say vertex 0) and tentatively say: "G1's vertex 0 = G2's vertex 0." Mark that G2 vertex as used.

Step 4: Move to the next vertex and repeat

Now try to match G1's vertex 1 to another unused vertex in G2.

Step 5: Check the mapping so far — the important check

Before accepting a match, ask: "For every pair of vertices I've matched so far, does the edge connection match on both sides?"

- If G1 has a connection between two vertices but the matched vertices in G2 don't have one (or the other way around) → this attempt is wrong.
- If everything lines up → keep going.

Step 6: If a match fails, go back and try a different one

This is the "backtracking" part — like trying a maze and turning back when you hit a dead end. Undo the last match, mark that G2 vertex as unused again, and try a different vertex instead.

Step 7: Keep repeating steps 3–6 for every vertex

Do this one vertex at a time until either:

- Every vertex in G1 has a valid match in G2 → the graphs ARE isomorphic (output TRUE), or
- Every possible combination has been tried and none worked → the graphs are NOT isomorphic (output FALSE)

Step 8: Show the final answer

Print whether they are isomorphic, and if yes, show exactly which vertex matched to which — like the output in your screenshot: 0→0, 1→1, 2→3, 3→